Sorting:

1) Selection Sort:

i/p →

0	1	2	3	4	5
13	46	24	52	20	9

o/p → 9, 13, 20, 24, 46, 52

Step 1 → 9 46 24 52 20 13 {swap(0,5)}

Step 2 → 9 13 24 52 20 46 swap(1,5)

⋮

Step 5 → 9 13 20 24 46 52

swap index 0 & min index (0, n-1)

swap index 1 & min index {1, n-1)

2 & {2, n-1]

⋮

n-2 [n-2, n-1]

imp

```
for (i=0; i<=n-2; i++) {

    min = i;
    for (j = i+1; j<=n-2; j++) {
        if (arr[j] < arr[min]) {
            min = j
        } swap([min], arr[i])
} }
```

Swap [·,·] → temp = arr[1]

arr[1] = arr[2]

arr[2] = temp

Time complexity :- $\frac{n^2}{2} + \frac{n}{2} \Rightarrow O(n^2)$ } Best case Avg

2) Bubble Sort :

{ Pushes the max to the last by adj swaps

i/p → 13 46 24 52 20 9

step 1 → 13, 24, 46, 20, 9, 52

[Step 1 max swaping
0 → n - 1

Step 2 → 13, 24, 20, 9, 46, 52 — 0 → n - 2

step 3 → 13, 20, 9, 24, 46, 52 — 0 → n - 3

4 → 13, 9, 20, 24, 46, 52 — ⋮

0 → 9, 13, 20, 24, 46, 52 → o/p

```
for [i = n-1 ; i >= 1 ; i--) {

    for ( j=0 ; j <= i-1 ; j++) {
        if [ a[j] > a [j+1] {
            Swap
        }
    }
}
```

Time complexity → $n + n-1 + n-2 \; --- + 2 = \frac{n \times (n+1)}{2}$

$= \frac{n^2}{2} + \frac{n}{2} = O(n^2)$ → worst
→ Avg.

optimized version

If array sorted in earlier steps then no need of running other steps. This helps optimization, TC

```
for (int i = n-1; i >= 1 ; i--) {
    int didswap = 0;
    for ( int j = 0; j <= i-1 ; j++) {
        if (arr[j] > arr[j+1]) {
            swap;
            didSwap = 1;
        } if (didswap == 0) {
            break; }
    } }
```

$O(N^2)$ → worst
→ Avg

T.C →

$O(N)$ → Best if array is already sorted.

Insertion Sort :

{ Takes an element & place it in its crct position

i/p 12 15 14 13

step-1 12 _ _ 15 14 13

as _ 12 ~~15~~ ~~14~~ 13 (14 15)

step 2 → 12 14 15 13

(13 14 15) 12 ~~14~~ ~~15~~ ~~13~~

Step 3 → 12 13 14 15 → o/p

```
for(int i = 1 ; i <= n-1 ; i++){
    int j = i ;
    while (j > 0 && arr[j-1] > arr[j]){
        Swap ;
        j--;
    }
}
```

T.C → $\frac{n \times (n+1)}{2} = \frac{n^2}{2} + \frac{n}{2}$ → $O(n^2)$ → Avg, → worst

if asendng order array → $O(n)$ → Best
(no swaps)